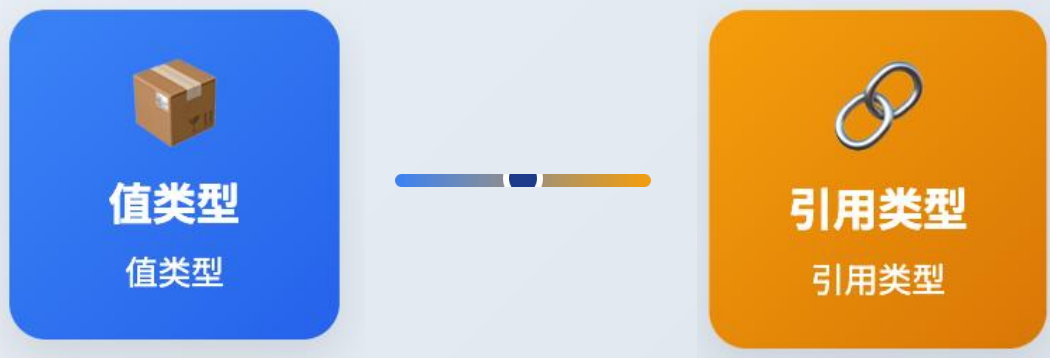


Solidity数据类型基础

智能合约开发的核心基石



目录

概述本次课程将涵盖的主要内容，包括数据类型分类、值类型、引用类型、操作符和类型转换等核心知识点。

数据类型分类

值类型详解

布尔类型

整数类型与运算

地址类型与功能

字节类型与字符串

引用类型详解

字符串类型与比较

枚举类型与优势

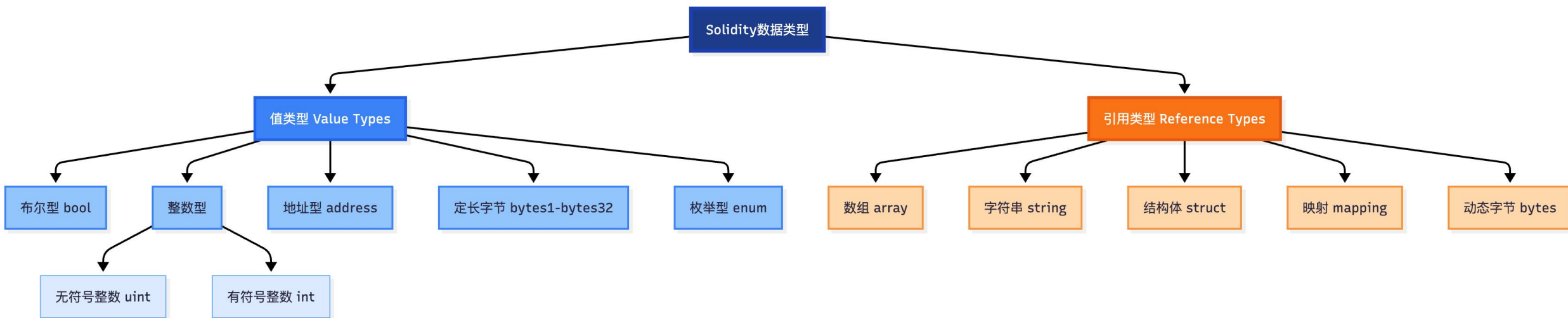
核心操作与转换

运算符总览与短路

类型转换与安全

数据类型分类

Solidity中的数据类型主要分为两大类：值类型和引用类型



值类型 vs 引用类型

Solidity中两大数据类型的核心区别

	 值类型	 引用类型
存储方式	直接存储数据本身	存储数据的内存地址（引用）
传递方式	传值：复制数据副本，互不影响	传引用：传递内存地址，共享同一数据
内存位置	通常存储在栈（Stack）或内存（Memory）中	通常存储在内存（Memory）或存储（Storage）中
修改影响	修改副本不影响原始数据	修改会直接影响原始数据

```
// 值类型的赋值
uint256 a = 100;
uint256 b = a; // b得到a的副本
b = 200;      // 修改b
// 结果：a还是100，b变成200，互不影响
```

```
// 引用类型的赋值（memory情况）
uint[] memory arr1 = new uint[](3);
arr1[0] = 1;
uint[] memory arr2 = arr1; // arr2指向同一个数组
arr2[0] = 100;             // 修改arr2
// 结果：arr1[0]也变成100，因为它们指向同一个数组！
```

布尔类型

布尔(bool)类型是Solidity中最基础的值类型之一

 **取值范围：** 只有true和false两个值

 **声明方式：** `bool isActive;`

 **常见操作：** 逻辑运算、比较运算

 **应用场景：** 条件判断、循环控制、状态标志

布尔类型特点

布尔类型在智能合约中主要用于条件判断和逻辑运算，是控制合约执行流程的基础。

布尔类型的存储成本

虽然布尔值只有两种状态（理论上只需要1位），但在EVM中，布尔类型实际上占用1个字节（8位）。

代码示例

```
bool isActive = true;    // 合约是否激活
bool isPaused = false;   // 合约是否暂停
bool isOwner = true;     // 是否是所有
if (isActive && !isPaused) { // “逻辑与”、“逻辑非”
    // 只有在激活且未暂停时执行
    doSomething();
}
// 比对判断
bool isOwner = (msg.sender == owner);
// 逻辑运算
bool result = isActive && isOwner;
// 循环判断
while (result ) {
    // 只要result 为true，就一直执行
}
```

正确用法

使用true和false字面量

注意事项

不能与其他类型隐式转换

整数类型概览

Solidity中的整数类型分为有符号和无符号两种，以8位为步长，从8位到256位

整数类型在Solidity中用于表示整数值，根据是否需要表示负数分为两大类：

+ 无符号整数 (uint)

只能表示非负整数（0和正整数），取值范围从0开始


uint8
0 - 255


uint16
0 - 65,535


uint32
0 - $2^{32}-1$


uint64
0 - $2^{64}-1$


uint256
0 - $2^{256}-1$

= 有符号整数 (int)

可以表示正整数、负整数和零，取值范围对称于零


int8
-128 - 127


int16
-32,768 - 32,767


int32
 $-2^{31} - 2^{31}-1$


int64
 $-2^{63} - 2^{63}-1$

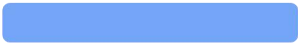

int256
 $-2^{255} - 2^{255}-1$

 **关键点：** Solidity中的整数类型以8位为步长递增，uint256是最常用的类型，因其与EVM原生字长匹配而效率最高。

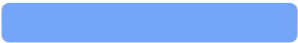
整数范围

Solidity提供多种整数类型，分为无符号整数（uint）和有符号整数（int）

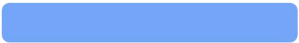
类型	范围	说明	类型	范围	说明
uint8	0 - 255	最小的无符号整数类型	int8	-128 - 127	最小的有符号整数类型
uint16	0 - 65,535	-	int16	-32,768 - 32,767	-
...	...	-	...	...	-
uint256	0 - 2^256-1	最大的无符号整数类型 ★最常用	int256	-2 - 2^255-1	最大的有符号整数类型



uint8



uint16



uint256



int8



int16



int256

uint256为什么最常用

在Solidity中，uint256是最常用的无符号整数类型，其主要原因如下：



EVM原生类型

以太坊虚拟机（EVM）内部处理256位数据效率最高，使用**uint256**可以避免额外的类型转换开销，从而降低Gas成本。



兼容性

与以太坊生态系统中的其他组件（如哈希值、地址等）保持一致，这些通常都是256位。使用**uint256**可以确保与其他组件的无缝兼容。



足够大的范围

uint256的取值范围足以满足绝大多数智能合约的数值需求，避免了溢出问题（在0.8.0版本之前）。

uint8

uint16

uint32

uint64

uint128

uint256

整数运算

Solidity支持多种整数类型，如uint8、uint256等，它们之间的运算规则与基本数据类型相同。

+ 加法：将两个数相加

- 减法：从第一个数中减去第二个数

***** 乘法：将两个数相乘

/ 除法：第一个数除以第二个数

% 取模：返回除法的余数

****** 幂运算：第一个数的第二个数次幂

```
uint8 a = 10; uint8 b = 5; uint8 c = 2; // 基本运算示例
uint8 sum = a + b; // 15
uint8 diff = a - b; // 5
uint8 product = a * b; // 50
uint8 quotient = a / b; // 2
uint8 remainder = a % b; // 0
uint8 power = a ** c; // 100
```

⚠ 注意：在Solidity 0.8之前版本，整数运算可能导致溢出
建议使用uint256类型，并在可能的情况下启用溢出检查。

溢出保护

Solidity 0.8版本前后整数溢出处理的对比

✖ 0.8之前 (红色)

```
uint8 a = 255;  
uint8 b = a + 1;  
// b 会变为 0 (溢出)
```

⚠ **问题：**静默溢出，导致不可预测的行为和安全漏洞。

✔ 0.8之后 (绿色)

```
uint8 a = 255;  
uint8 b = a + 1;  
// 交易会回滚 (revert)
```

🛡 **解决方案：**自动检查溢出，若发生则回滚交易，确保数据完整性。

💡 **核心改进：** Solidity 0.8.0引入了对整数运算的默认溢出检查，极大地提升了合约的安全性。

地址类型

Solidity特有的20字节以太坊账户地址类型

address

❌ 无法接收以太币

👁️ 只读操作可用

</> 示例: `address owner = msg.sender;`

```
address owner = msg.sender;
```

address payable

↔️ 可以接收以太币

➡️ 具有`transfer()`方法

</> 示例: `address payable receiver = payable(0xAbC...);`

```
address payable receiver = payable(0xAbC...);
```

i 常用功能: `address(0)` - 零地址, `msg.sender` - 调用者地址, `.balance` - 余额查询

地址功能

详细说明地址类型的常用功能，如余额查询、转账操作等

常用功能

 `address(0)`
表示零地址，常用于初始化或检查空地址

 `msg.sender`
表示当前函数调用的发起者地址

 `.balance`
查询地址的以太币余额

 `.transfer(amount)`
向地址发送指定数量的以太币（仅限 `address payable`）

代码示例

```
// 声明地址变量
address owner = msg.sender;
address payable receiver = payable(0xAbC...); // 示例地址

// 查询余额
uint256 balance = owner.balance;

// 转账操作
receiver.transfer(1 ether); // 发送1 ETH
```

⚠ 注意:

`address`类型无法接收以太币，而`address payable`类型可以接收并具有`transfer`方法

字节类型

Solidity中的字节类型分为定长和变长两种形式，各有不同的应用场景。

定长字节数组

- **bytes1 到 bytes32**
 - 固定长度的字节序列
 - bytes1存储1个字节，bytes32存储32个字节
 - 值类型 (value type)
 - 存储在栈 (Stack) 或内存 (Memory) 中

```
bytes32 hashValue = keccak256(abi.encodePacked("hello"));
bytes4 selector = bytes4(keccak256("transfer(address,uint256)"));
```

✔ 适用于：存储哈希值、固定长度二进制数据

变长字节数组

- **bytes**
 - 可变长度的字节序列
 - 存储任意长度的二进制数据
 - 引用类型 (reference type)
 - 存储在内存 (Memory) 或存储 (Storage) 中

```
bytes memory data = abi.encodePacked("Hello", " ", "World");
bytes memory encoded = abi.encode(data, uint256(42));
```

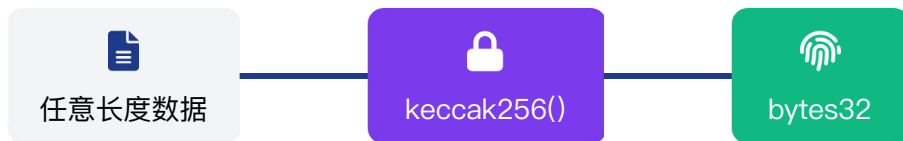
✔ 适用于：存储可变长度二进制数据、动态内容

特性		bytes1-bytes32	适用场景
存储方式	直接存储数据	存储数据的内存地址	固定长度数据使用bytes1-bytes32
传递方式	传值	传引用	可变长度数据使用bytes

bytes32与哈希函数

keccak256哈希函数

keccak256函数接受任意长度的输入数据，并输出一个固定长度的32字节哈希值。



示例代码

```
function getHash(string memory text)
    public pure returns (bytes32)
{
    return keccak256(bytes(text));
}
```

主要用途



存储哈希值

keccak256的输出，用于数据完整性验证或作为唯一标识符



存储签名

区块链交易中的数字签名通常是固定长度的字节序列



存储唯一标识符

例如，某些代币的ID或特定数据的哈希表示

应用场景



数据完整性验证



身份验证



代币标识



Merkle树构建

字符串类型

变长UTF-8编码的字节数组，作为引用类型存储

特性

-  **可变长度：** 存储任意长度的文本数据
-  **UTF-8编码：** 存储UTF-8格式的文本
-  **引用类型：** 存储数据的内存地址（引用）
-  **Gas成本：** 操作字符串消耗更多Gas
-  **比较特殊：** 不能直接使用==或!=运算符

存储方式

引用类型特点

字符串存储在内存或存储中，传递时传递的是数据的引用而非实际数据副本。修改字符串可能影响所有引用它的变量。

字符串比较需特殊处理：

```
</> 使用keccak256(abi.encodePacked(_s1)) ==  
keccak256(abi.encodePacked(_s2))
```

</> 字符串比较

⚠ **问题：** 在Solidity中，字符串是比较它们的内存地址而非内容。

✖ 错误做法

```
function compareStringsIncorrect(string memory _s1, string
memory _s2) public pure returns (bool) {
    return _s1 == _s2; // 比较内存地址，而非内容
}
```

❗ 直接使用 `==` 比较字符串变量，实际上是比较它们在内存中的地址

✓ 正确做法

```
function compareStringsCorrect(string memory _s1, string
memory _s2) public pure returns (bool) {
    return keccak256(abi.encodePacked(_s1)) ==
keccak256(abi.encodePacked(_s2));
}
```

❗ 通过 `keccak256(abi.encodePacked(...))` 将字符串内容转换为哈希值进行比较

💡 **原因：** 字符串是引用类型，存储在内存或存储中，比较操作符默认比较的是引用而非内容。

枚举类型

基本概念

- ✓ 用户自定义的有限集合类型
- ✓ 创建一组命名的常量
- ✓ 提高代码可读性和可维护性
- ✓ 避免使用魔术数字或硬编码值

枚举的优势

👍 代码更易读、更安全、更易维护

示例代码

```
enum State { Pending, Active, Closed }  
  
State currentState = State.Pending;  
  
// 检查状态  
if (currentState == State.Active) { // 执行操作  
  
}
```

Pending → Active → Closed

枚举优势

枚举类型（enum）为智能合约开发带来多项优势，提高代码质量与安全性



提高可读性

使用有意义的名称代替魔术数字，使代码自文档化



增强可维护性

集中定义常量，便于统一修改与扩展



类型安全性

编译时检查枚举值的有效性，防止非法状态

对比示例：

不使用枚举：

```
// 魔术数字，难以理解  
if (status == 0) { // 0代表什么? }
```

使用枚举：

```
enum Status { Pending, Active, Closed }  
Status s = Status.Pending;  
// 清晰的语义，避免错误
```

运算符总览

Solidity提供丰富的运算符，可对不同类型的数据进行操作

算术运算符

+

-

*

/

%

**

加法、减法、乘法、除法、取模、幂运算

比较运算符

==

!=

>

<

>=

<=

等于、不等于、大于、小于、大于等于、小于等于

逻辑运算符

&&

||

!

逻辑与、逻辑或、逻辑非（短路求值）

位运算符

&

|

^

~

<<

>>

按位与、按位或、按位异或、按位非、左移、右移

短路运算

💡 在Solidity中，逻辑运算符`&&`（逻辑与）和`||`（逻辑或）支持

这意味着如果第一个操作数已经足以确定整个表达式的结果，那么第二个操作数将不会被评估。

这一特性在节省Gas方面尤为重要，因为它避免了不必要的计算。

// 逻辑与 (&&) 示例

```
A && B;
```

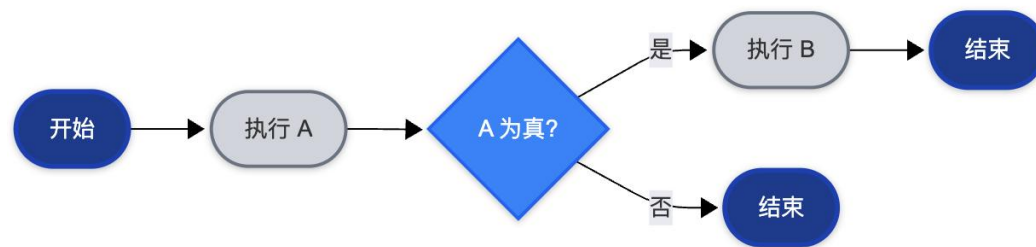
// 如果 A 为 false，则 B 不会被执行

// 逻辑或 (||) 示例

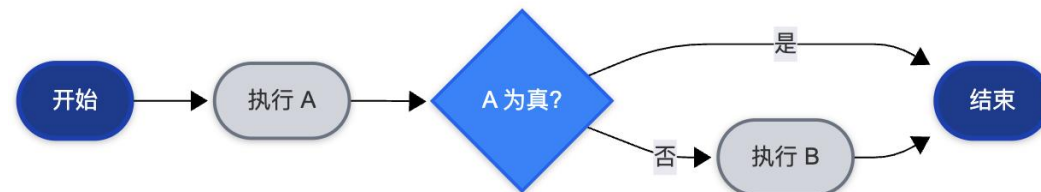
```
A || B;
```

// 如果 A 为 true，则 B 不会被执行

逻辑与 (&&) 短路流程图



逻辑或 (||) 短路流程图



类型转换

Solidity中的类型转换机制

→ 隐式转换

较小类型可以安全地转换为较大类型而不会丢失信息

```
uint8 → uint256
```

- ✓ 编译器自动完成
- ✓ 安全，不会丢失数据

↔ 显式转换

较大类型转换为较小类型，可能导致数据截断或溢出

```
uint256 → uint8
```

- ⚠ 开发者需明确使用类型转换操作符
- ⚠ 可能导致数据丢失

🛡 安全转换建议

```
uint8 a = uint8(x);
```

- ✓ 在转换前进行范围检查，确保值在目标类型的有效范围内

安全转换

显式类型转换前进行范围检查，防止数据截断或溢出

为什么需要安全转换？

⚠ 显式转换可能导致数据截断或溢出

✅ 范围检查是避免潜在漏洞的最佳实践

类型转换安全检查流程

- 1 接收待转换的值
- 2 使用require语句检查范围
- 3 执行安全转换

❌ 错误做法：未检查范围的显式转换

```
function unsafeConvert(uint256 _value) public pure returns (uint8) {  
    // 如果 _value > 255，这里会发生溢出  
    return uint8(_value);  
}
```

✅ 正确做法：进行范围检查的显式转换

```
function safeConvert(uint256 _value) public pure returns (uint8) {  
    require(_value <= type(uint8).max, "Value exceeds uint8 max");  
    return uint8(_value);  
}
```

地址转换

详细介绍地址类型的转换方法，包括普通地址和可支付地址之间的转换，以及与其他类型的互转。

 Solidity中有两种地址类型：

address：无法接收以太币

address payable：可以接收以太币

↔ 地址类型转换

// 地址到可支付地址的显式转换

```
address payable receiver = payable(address(0xAbC...));
```

// 可支付地址到普通地址的显式转换

```
address normal = address(receiver);
```

↔ 与其他类型转换

// uint256到address

```
address addr = address(uint160 value);
```

// msg.sender到address payable

```
address payable sender = payable(msg.sender);
```

⚠ 安全提示

进行地址类型转换时，始终确保目标地址可以接收ETH，特别是从外部输入转换时，务必进行有效性检查。

类型选择建议

提供实用的数据类型选择建议，帮助开发者在实际项目中根据不同场景选择合适的数据类型。

📊 整数类型选择

- ✅ 优先使用最小合适的整数类型（如uint8、uint16）
- ⚠️ 确保存在负数时才使用int类型
- 💡 对于大多数通用场景，uint256是安全且高效的选择

📍 地址类型选择

- ✅ 如果需要接收ETH，务必使用address payable
- ℹ️ address类型无法接收ETH，但可以用于一般账户表示

📄 字符串与字节数组

- 📄 对于短文本或固定长度标识符，考虑使用bytes32
- ✅ 对于变长文本，使用string类型
- 🔗 字符串比较务必使用keccak256哈希

🔑 枚举与类型转换

- ✅ 当变量只有有限的预定义状态时，使用enum提高可读性
- ⚠️ 避免不必要的类型转换，减少运行时开销
- 🛡️ 进行显式类型转换时，始终先检查范围以防止溢出

知识总结

回顾本课的关键知识点，巩固对Solidity数据类型的理解

数据类型分类

-  **值类型**: bool、int/uint、address、bytes1-32、enum (蓝色系)
-  **引用类型**: array、string、struct、mapping、bytes (橙色系)

值类型特性

-  赋值时创建完整副本，独立存储
-  修改副本不影响原始数据
-  通常存储在栈或内存中

引用类型特性

-  存储数据的内存地址 (引用)
-  传递内存地址，共享同一数据
-  通常存储在内存或存储中

运算符与类型转换

-  算术、比较、逻辑和位运算符
-  隐式转换 (安全) 与显式转换 (谨慎)
-  安全转换需先检查范围

课后作业

为了巩固本课所学知识，请完成以下任务：

1 实践编码

编写一个简单的Solidity合约，包含至少一种值类型和一种引用类型，并实现它们之间的基本操作和一次类型转换。

2 Gas成本分析

尝试使用不同大小的`uint`类型存储相同的数据，并观察其Gas消耗差异。

3 溢出测试

编写代码测试Solidity 0.8版本前后整数溢出的行为。

课程回顾

简明扼要地回顾本节课的主要内容，包括数据类型、运算规则和安全实践，作为课程的总结。

数据类型分类

Solidity数据类型分为两大类：值类型（，蓝色系）和引用类型（，橙色系），它们在存储和传递方式上有根本区别。

核心引用类型

引用类型包括数组、字符串、结构体、映射和字节数组，它们在内存或存储中操作，传递的是数据的引用而非副本。

运算符与转换

Solidity支持算术、比较、逻辑和位运算符，逻辑运算符具有短路求值特性。类型转换分为隐式（安全）和显式（需谨慎）两种形式。

安全实践

进行显式类型转换时务必进行范围检查，避免溢出或截断。使用uint256作为默认整数类型，利用Solidity 0.8+的溢出保护特性提升安全性。

下节预告

下一课将深入探讨Solidity中的函数与修饰符，学习如何构建可重用、安全的合约逻辑。

下一课主题：函数与修饰符

在智能合约中组织和重用代码的关键机制

函数定义与调用



修饰符基础



可见性修饰符



访问控制修饰符

纯函数与视图函数

循环与条件修饰符

→ 准备好探索函数与修饰符的奥秘了吗？